

Q1:

1. Using req.body in a GET request
 - Location: `app.get("/api/users/logout", (req, res) => { ... })`
 - Description: The req.body property is being used in a GET request. GET requests should not have a body; they are intended to retrieve data without causing any side effects.
 - Fix: Use query parameters or URL parameters to pass the userId. For example, use req.query.userId or include the userId in the URL path.
2. Incorrect use of URL parameters
 - Location: `app.post("/api/users/:userId/images", (req, res) => { ... })`
 - Description: The endpoint is using a URL parameter :userId, but it is not being accessed correctly in the code. The code uses req.params.userId, but it should be req.body.userId for consistency and clarity.
 - Fix: Ensure that userId is consistently accessed either from req.params or req.body. Using URL parameters is fine, but make sure it is accessed correctly.
3. Missing error handling for database operations
 - Location: `app.get("/api/users/images", (req, res) => { ... }) / app.post("/api/users/:userId/images", (req, res) => { ... })`
 - Description: The code does not handle potential errors that could occur during database operations such as getUserById and getImagesByUser.
 - Fix: Add try-catch blocks around database operations to handle exceptions and send appropriate error responses.
4. Potential race condition with totalPosts
 - Location: `app.post("/api/users/:userId/images", (req, res) => { ... }) / app.get("/api/images/totalPosts", (req, res) => { ... })`
 - Description: The variable totalPosts is being incremented without any synchronization, which could lead to race conditions in a concurrent environment.
 - Fix: Use atomic operations or a database counter to safely increment and retrieve the total number of posts.
5. Inconsistent endpoint naming and structure
 - Location: `app.get("/api/users/logout", (req, res) => { ... }) / app.get("/api/images/totalPosts", (req, res) => { ... })`
 - Description: The endpoint naming and structure are inconsistent. For example, logout is nested under /api/users/, while totalPosts is directly under /api/images/.

- Fix: Standardize the endpoint structure. For example, use `/api/users/logout` and `/api/users/images/totalPosts` or `/api/images/totalPosts`.

Q2

GET

Path: `/Profile`

Method: GET

Body: (No Body Need)

Responses:

- **200:** OK
 - `{name: req.session.user.preferName, defaultName: req.session.user.username}`.
- **401:** Unauthorized
 - `{name: 'Unauthorized', 'defaultName:Unauthorized' }`.
- **500:** err
 - `{name: 'err', defaultName: 'err' }`.

POST

Path: `/api/createFlashCard`

Method: POST

Body: Content: {

```
fornt: {
  title: req.body.frontTitle,
  content: req.body.frontContent,
},
back:{
  title: req.body.backTitle,
  content: req.body.backContent,
}
}
```

Responses:

- **200:** OK
 - `{message: 'Successfully Created'}`.
- **500:** err
 - `{message: 'Failed to Create'}`.

DELETE

Path: /api/deleteFlashCard

Method: DELETE

Body: Content: {

id: id

}

Responses:

- **200:** OK
 - {message: 'FlashCard deleted successfully'}.
- **500:** err
 - {message: 'Error deleting FlashCard'}.

PATCH

Path: /MindMap/AutoSave

Method: PATCH

Body: Content: {

id: id,

content: {

 nnodeData: Object,

 linkData: Object,

 direction: Int,

 theme: Object

}

}

Responses:

- **200:** OK
 - {message: 'Update successful'}.
- **404:** Not Found
 - {message: 'Document not found' }.
- **500:** err
 - {message: 'Error deleting FlashCard'}.

Q3

Describe a scenario where running this endpoint could be harmful - If the database contains private images, exposing all images through this endpoint may cause a privacy violation. - **Propose a solution to address this issue -**

We need **authorization checks** to ensure that only users with the appropriate permissions can access these images

Pseudocode

```
GET /api/images/
function (req, res){
  user = req.session.user
  public_images = db.images.find({public: true});
  try{
    if (user.role !== 'admin'){
      return res.status(200).json(public_images);
    }else {
      all_images = db.images.findAll();
      return res.status(200).json(all_images);
    }
  }catch(error){
    console.error('Error');
    return res.status(500).json({ error: 'Error' });
  }
}
```

This will check the role of the logged in user and if the role is not admin then only res.json will be exposed as annotations, only users with admin role will have access to all images.